General presentation

Content

- Database principles
- Database design
- SQL primer with MySQL
- MySQL access from Node.js
- ORM’s

Content

- LDAP and identity management

Content

- MySQL security
- MongoDb
- Mongo access from Node.js
- ODM’s
- MongoDb security
- MongoDb distributed architectures

Content

- Redis
- Using Redis from Node.js
- Neo4j
- Using Neo4j from Node.js
- Advanced application architectures

General discussion

- What are databases?
- Database evolution
- Database structure
- Entity-value models
- Entity-attribute-value models
- The classical relational model
- Database normalization and denormalization
- Alternate models

Database design

Usefulness of database design

- Moving (part of) the checks for data integrity from the application realm to the database
- Making the database model easier to explain and maintain

Anomalies in data

- Anomaly - situation where data that should be handleable separately cannot be treated as such; this leads to situations where the real life circumstances automated by the system cannot be reflected in it
- Types of anomalies
  - Insert anomalies - data cannot be inserted until other data is available
  - Update anomalies - updating one or more data items creates inconsistencies within the data
  - Delete anomalies - data cannot be deleted without deleting other data, which should be kept

Normalization

- Restructuring the data by splitting into additional relations so that anomalies are minimized
- Goals
  - Decrease redundancies
  - Eliminate some/all anomalies
  - Increase model explainability
  - Make the data representation neutral to query statistics (it should not favor particular queries)
- Ultimately the overarching goal of normalization is to have each attribute in a relation specify something about the key and only that

Denormalization

- Goal - increasing speed of information retrieval
- Denormalization and normalization represent a trade-off (there is no free lunch)

Terms

- Relation vs table
- Attribute vs column
- Row vs tuple
- Relational schema vs database
- Attribute domain

Principles of normalization

- A normal form is a set of conditions that must be respected by the defined relations
- Normalization functions in stages, bringing the relations in consecutive, stricter and stricter normal forms
- Higher normal forms are seldom used, since the anomalies they correct are subtle and not necessarily encountered in the data model being designed
- Additionally, higher normal forms tend to lead to a large number of tables (table explosion)
- **A relational schema is in a superior normal form is also in all inferior forms. For example a data model in 3NF is automatically in 2NF**

Normal forms (in order of strictness)

- 1NF - first normal form
- 2NF - second normal form
- 3NF - third normal form
- EKNF - elementary key normal form
- BCNF - Boyce-Codd normal form
- 4NF - fourth normal form
- 5NF - fifth normal form
- DKNF - domain-key normal form
- 6NF - sixth normal form

Concepts

- A **relation** is a set of attributes
- A **superkey** is a set of attributes that uniquely identifies a row
- A **candidate** key is a set of attributes that have a unique value for each row; candidate keys are minimal superkeys, in that no attribute can be taken out while still uniquely identifying every row
- A **dependency** means some set of attributes specifies something about another set of attributes
- A dependency is **trivial** if the second set is included in the first

1NF conditions

- Atomic value - value of a column/row intersection cannot be split into smaller relevant values
- No repeating groups - each row/column intersection contains at most one value
- Uniqueness - each tuple is unique
- **Simplified:**
  - There are no attributes which function as a list of values
  - Each piece of information is found in one and only one attribute

1NF continued

- 1NF is equivalent to adherence to the relational model
- Additional conditions stemming from that (Date):
  - There's no top-to-bottom ordering to the rows
  - There's no left-to-right ordering to the columns
  - There are no duplicate rows
  - Every row-and-column intersection contains exactly one value from the applicable domain (and nothing else)
  - All columns are regular (there are no special, hidden components to rows)

2NF

- Respects 1NF
- No functional dependencies on a subset of a candidate key (generally on the primary key, if the designer knows other keys will not be used); basically, no partial dependencies
- **Simplified:**
  - No attribute depends on only part of the primary key (all keys actually but see above)

3NF

- Respects 2NF
- Every non prime attribute is non-transitively dependent on every candidate key (no transitive dependencies)
- **Simplified:**
  - No attribute depends on a non key attribute, which in turn depends on the key
  - Attributes depend directly on the key
- Reasonably high guarantee that **Every non-key attribute "must provide a fact about the key, the whole key, and nothing but the key."**

Superior normal forms

- Tend to refer to subtle differences, which may or may not cause anomalies in a given data model

EKNF

- A table is in EKNF if and only if all its elementary functional dependencies begin at whole keys or end at **elementary** key attributes

BCNF

- A relational schema R is in BCNF if and only if all of its dependencies, are either trivial or dependencies on a superkey

4NF

- Respect 3NF
- Every non-trivial multivalued dependency in the table is a dependency on a superkey
- There are no multivalued dependencies

5NF

- Respects 4NF
- Every non-trivial join dependency in that table is implied by the candidate keys
- A join dependency refers to the fact that splitting a table into multiple tables and then performing a join on them can recreate the original table
- Last normal form relating to data redundancy

DKNF

- Refers to the data domains, not the organization in relations
- A schema is in DKNF if all relations only have domain constraints and key constraints and no other constraints

6NF

- Respects 5NF
- Contains no non trivial join dependencies

Sql

- A short refresher on SQL -

Database programming

Native RDBMS access with node.js

Principles

- In nodejs access to a relational database is done via a native database driver
- This is similar to how database access would be done in other languages, but without an overarching standardized library
- The native mysql driver can included via its npm package, mysql2
- The driver supports promise style programming, and thus async/await
- ! A refresher on promises

Getting a connection

- The driver provides connections via the mysql.createConnection method
- For any connection to any relational database in any language three things must be provided
  - Username
  - Password
  - Database name (this can be in the form of a combination of server and database)
- Additionally, the port can be provided if it is different to the default one (i.e. 3306)
- Connections can be closed with the destroy method of the connection object

Getting a connection

```
const connection = await mysql.createConnection({
 host: 'localhost',
 database: 'ismv4',
 user: 'app1',
 password: 'welcome123'
})

await connection.destroy()
```

Handling errors

- When creating connections and indeed for any operation, if using the async/await programming style, it is sufficient to wrap the operations in a try/catch block
- Whether the operations are successful or not the connection should be closed on a finally block, as connections are rare resources

Handling errors

```
try {
 const connection = await mysql.createConnection({
   host: 'localhost',
   database: 'ismv4',
   user: 'app1',
   password: 'welcome123'
 })
} catch (error) {
 console.warn(error)
} finally {
 await connection.destroy()
}
```

Connection pooling

- Creating connections is expensive
- One way of tackling this problem is to keep a set of connections open and reuse them for multiple operations
- Such a set of connections is called a connection pool
- A connection pool has to contain the same elements as a connection (database name, username, password) as well as defining:
  - How many connections are in the pool
  - Whether, when the pool is fully used, connections are rejected outright or enqueued in a waiting queue
  - How many unused connections should be kept open
  - What is the connection timeout for a connection on which no operations have been performed in a while

Connection pooling

```
const pool = mysql.createPool({
 host: 'localhost',
 database: 'ismv4',
 user: 'app1',
 password: 'welcome123',
 waitForConnections: true,
 connectionLimit: 10,
 maxIdle: 10, // max idle connections, the default value is the same as `connectionLimit`
 idleTimeout: 60000, // idle connections timeout, in milliseconds, the default value 60000
 queueLimit: 0
})
await pool.end()
```

CRUD operations

- Generally there are two kinds of operations based on their return values
  - Ones that return a number of affected rows
  - Ones that return a table like structure
- All commands can be sent as strings to the database via the network protocol exposed by the database server
- Command return an array of the rows in the result and the fields of the schema on which the command was run
- Commands are run with the connection.query method
- For insert, update and delete we get a result detailing the number of rows affected by the command
- For select we get an array of objects representing the result of the query

Crud operations - insert

```
let [rows, _] = await connection.query('insert into
students (first_name, last_name, telephone, email) value
("jane", "smith", "222-222", "jane4@etc.com")');
```

Crud operations - update

```
[rows, _] = await connection.query('update students set
email = "j@etc.com" where email="jane4@etc.com" ');
```

Crud operations - delete

```
[rows, _] = await connection.query('delete from students
email="j@etc.com" ');
```

Crud operations - queries

```
[rows, _] = await connection.query('select * from students');
[rows, _] = await connection.query('select * from students
where first_name="jane"')
[rows, _] = await connection.query('select * from students
where first_name="jane" limit 10,100')
```

A short comment on SQL injection

- Input coming directly or indirectly from the user should always be considered potentially hostile
- If we build commands via string concatenation, the user can provide input to derail the original intent of the command
- When constructing an SQL command based on user input, we should take care to either sanitize the input (complicated and prone to error if we aim to maintain user intent) or use prepared statements
- Prepared statements check that the user input is not executable content

Injection example

```
let queryString = 'select last_name from students where email='

let regularUserInput = 'jane@etc.com'
let hostileUserInput1 = "' or 1=1;#"
let hostileUserInput2 = "' or 1=1 union select user from
mysql.user;#"

let [rows, _] = await connection.query(queryString + "'" +
hostileUserInput2 + "'")
```

Prepared statements

- A prepared statement is basically a template into which values are plugged in at each execution
- They can be represented at application level or at database level (as anonymous stored procedures)
- In mysql2 prepared statements can be created with connection.prepare and closed with statement.close
- They can also be created on demand with connection.execute
- The command provided for a prepared statement has “gaps” noted with ? into which values can be plugged in

Prepared statement

```
await connection.execute('insert into students (first_name, last_name, telephone, email) value (?, ?, ?, ?)', ['clara', 'smith', '111', 'clara@etc.com'])

await connection.execute('delete from students where first_name = ?', ['clara']);
```

Calling stored procedures

- Stored procedures can be executed with the query method whilst passing parameters in an array (if the stored procedure has parameters)
- The results of the stored procedure execution are then returned by query (after the promise is resolved)

Calling stored procedures

```
const inputString = 'Hello'

await connection.query('CALL concatenate_with_date(?,
@output)', [inputString])

const [rows] = await conn.query('SELECT @output AS result')

console.log('Result: ', rows[0].result)
```

Database programming

ORM access with node.js

Refresher on ORM’s

- ORM’s (Object Relational Mappers) bridge the representation gap between the object-oriented and relational models
- They constitute an additional layer of abstraction increasing productivity, but sometimes decreasing control
- They allow us to offload some of the responsibilities of validation to a standardized mechanism
- They are heavily based on the convention-over-configuration approach, that is to say, we need only define that which doesn’t respect the convention
- ORM’s allow us to use the same code for multiple RDBMS

Refresher on ORM’s

- The most used ORM for nodejs is sequelize
- Apart from sequelize itself, we must include in our project one of the supported native drivers
- Sequelize is based on a strong convention, but we can configure our particular needs

Getting a connection

- The first step in using an ORM is getting a connection, just like for the native driver
- Since an ORM can store data over a variety of RDBMS’s we need to specify which one we are using with the dialect configuration option
- When creating a connection we can also specify differences from the convention at the general level

Getting a connection

```
import Sequelize from 'sequelize'

const sequelize = new Sequelize({
 dialect: 'mysql',
 host: '172.18.0.2',
 username: 'app1',
 password: 'welcome123',
 database: 'ism_v4'
})

await sequelize.authenticate()
sequelize.close()
```

Handling errors

- Sequelize also functions on an async/await approach, which means we can handle errors via a try/catch/finally block

Handling errors

```
try {
 await sequelize.authenticate()
 console.log('we are connected')
} catch (error) {
 console.warn(error)
} finally {
 sequelize.close()
}
```

Entities/persistent objects

- ORM’s function by defining models/entities (which are type definitions for persistent models)
- Typically, models don’t just contain the data, but also support methods to interact with it
- A model with the additional methods is called a hydrated model
- A model object that just contains the data is called dehydrated or lean

Defining single entities

- Given a connection, entities can be defined with sequelize.define
- A model can be defined in a minimalistic manner, with the rest being filled in by a convention
- Models can be defined in a simplified form by supplying their type
- A mode advanced definition allows us to make them non nullable, validate them, as well as define differences from the convention

Defining single entities

```
const Author = sequelize.define('author', {
 name: Sequelize.STRING,
 email: Sequelize.STRING
})
```

Defining single entities

```
const Student = sequelize.define('student', {
 firstName: {
   type: Sequelize.STRING,
   allowNull: false,
   columnName: 'first_name'
 },
 lastName: {
   type: Sequelize.STRING,
   allowNull: false,
   columnName: 'last_name'
 },
 phone: {
   type: Sequelize.STRING,
   allowNull: false,
   columnName: 'telephone',
   validate: {
     is: /^\+40\d{9}$/
   }
 },
 email: {
   type: Sequelize.STRING,
   allowNull: false,
   columnName: 'email',
   unique: true,
   validate: {
     isEmail: true
   }
 }
}, {
 tableName: 'students',
 timestamps: false
})
```

Defining one-to-one relationships

- One to one relationships can be defined by using the hasOne method on an entity
- The relationship can be defined in both directions
- When creating a relationship the model is decorated with additional methods which allow for queries on the associated entity

Defining one-to-one relationships

```
Restaurant.hasOne(Menu)

Menu.belongsTo(Restaurant)
```

Defining one-to-many relationships

- One to many relationships can be created with hasMany or belongsTo methods (from the two directions)
- When creating a relationship the model is decorated with additional methods which allow for queries on the associated entity

Defining one-to-many relationships

```
Restaurant.hasMany(Location)

Location.belongsTo(Restaurant)
```

Defining many-to-many relationships

- Many to many relationships take two forms
  - Either the association has no semantic content apart from matching a record on the right to a record on the left
  - Or the association does have semantic content
- In the first situation, the through configuration parameter in the options parameter of sequelize.define can be used (on both sides of the relationship)
- In the second case, the many to many relationship is actually transformed into two different one to many relationships

Defining many-to-many relationships

```
Menu.belongsToMany(MenuItem, { through: 'MenuItemMapping' })
MenuItem.belongsToMany(Menu, { through: 'MenuItemMapping' })
```

Creating tables from models

- There are two directions in which models can be associated to tables
- For a new application connection.sync can be used
  - Tables can be deleted with recreated with the parameter force set to true
  - Tables can be altered with the parameter alter set to true
  - Only new tables can be created (while existing ones are ignored) with the parameter force set to false
- For legacy applications, it is more difficult to backup the data, create the models and then migrate the data to a new structure
- As such, we need to veer away from the convention and specify the existing table and column names via the options to sequelize.define and the fields themselves

Creating tables from models

```
try {
 // won't do anything if the table already exists
 // can be circumvented with {force: true}
 await sequelize.sync({ force: true })
 console.log('created')
} catch (error) {
 console.warn(error)
} finally {
 sequelize.close()
}
```

Crud operations - insert

- A record can be inserted in multiple ways
  - A persistent object can be created in memory and then the model.save method can be called
  - Model.create can be used to create the object directly
- There are two stages to insertion
  - First the object is created in memory and validated
  - Then the corresponding SQL is generated and executed against the database
- Validations provided by sequelize are more extensive than the ones supported by a particular RDBMS

Crud operations - insert

```
 const restaurant1 = await Restaurant.create({ name:
'Restaurant 1', openingHour: '10:00', closingHour: '22:00'
})
 const restaurant2 = await Restaurant.create({ name:
'Restaurant 2', openingHour: '11:00', closingHour: '23:00'
})
```

Crud operations - update

- There are two ways to perform updates
  - Updates can be performed on one loaded record
  - Updates can be performed on the model itself, thus affecting all records
- If updating one record, the object has to be first loaded into memory, modified and then saved with record.save
- A special fields option can be passed to avoid updating parts of the model which should not be changed by the user
- When updating multiple records, the update method on the model itself is run with a query and an updated version of some/all of the fields in each record

Crud operations - update

```
// Modify the name of the first restaurant
 await restaurant1.update(another, {fields: ["name"]})

// Change everyone without a last name to "Doe"
await User.update({ lastName: "Doe" }, {
 where: {
   lastName: null
 }
})
```

Crud operations - delete

- There are two ways to perform deletes
  - Deletes can be performed on one loaded record
  - Deletes can be performed on the model itself, thus affecting all records
- If deleting one record, the object has to be first loaded into memory, then deleted with record.destroy
- When deleting multiple records, the destroy method on the model itself is run with a query

Crud operations - delete

```
// Delete everyone named "Jane"
await User.destroy({
 where: {
   firstName: "Jane"
 }
});
```

Crud operations - queries

- In sequelize, queries can be run with the find, findByPk, findOne, findOrCreate and findAndCountAll
- All of these methods can be called on a model
  - The find method returns all records matching the query condition
  - The findByPk returns one record baked on a primary key
  - The findOrCreate method returns an object always returns the object, by creating it if it does not exist
  - The findAndCountAll method returns an object with the rows and count keys (useful for paging through data); for paging, count will contain the number of all records matching the query, not only those included in the rows array

Selection conditions in queries

- The sequelize find method receives an object with multiple options
- The where option can be used to supply a query which works by applying a mask to all records
- The list of query operators in sequelize is more extensive than what some RDBMS's support, but their availability depends on the native driver
- Comprehensive list of query operators
  - https://sequelize.org/docs/v6/core-concepts/model-querying-basics/

Selection conditions in queries

```
let people = await Person.findAll({
 where: {
   salary: {
     [Op.gt]: 1000,
     [Op.lt]: 3000,
   }
 },
 raw: true
})
```

Projection

- Projection can be achieved via the attributes parameter to a sequelize query
- If in the projection part we want to apply an aggregation function or alias a column, instead of sending a column name we will send an array containing the functions (called with sequelize.fn) and the alias (in that order)

Projection

- Projection can be achieved via the attributes parameter to a sequelize query
- If in the projection part we want to apply an aggregation function or alias a column, instead of sending a column name we will send an array containing the functions (called with sequelize.fn) and the alias (in that order)

Projection

```
let people = await Person.findAll({ raw: true })
people = await Person.findAll({
 where: {
   name: 'john',
   age: 24
 },
 attributes: ['name', 'salary']
})
```

Sorting and grouping

- Sorting can be performed with the order parameter
- The order will be sent as an array of arrays, each inner array containing the name of the field and the order
- These will be used for sorting in the order in which they appear
- Aggregate functions can be used in the order by calling the function in the first position and specifying the order in the second position
- Grouping can be done by using the group parameter

Sorting and grouping

```
Foo.findOne({
 order: [
   // will return `name`
   ['name'],
   // will return max(`age`)
   sequelize.fn('max', sequelize.col('age')),
 ]
})
Project.findAll({ group: 'name' });
```

Paging

- In order to not return all records which match a particular query, paging can be used
- Just like in SQL, paging is done with the parameters limit and offset

Paging

```
const students = await Student.findAll({
 offset: page * limit,
 limit: limit
});
```

Joins

- The most powerful mechanism in SQL are joins
- By default dependent models are not included with their parent (this is called lazy loading)
- We can, however, load models eagerly
- Joins can be done by including an additional associated model with the include parameter
- Multiple models can be included
- Models can be included in depth (instead of just specifying the model name we will have to provide a more complex definition)

Joins

```
let results = await Restaurant.findAll({
 where: { id: 1 },
 include: [Menu]
})
const book = await Book.findByPk(req.params.bid, {
 include: Chapter
})
```

Including models in depth

- Models can be included in depth by passing to include an object containing the model key
- This object can also have its own where, attributes etc. clauses as well as its own include clause

Including models in depth

```
let results = await Restaurant.findAll({
 where: { id: 1 },
 include: [
   {
     model: Menu,
     include: [{ model: MenuItem }]
   }
 ]
})
```

Auto-generated methods

- Joins/include allow us to include dependent entities alongside their parents with a single query
- If, however, having already loaded the parent we need to load the children separately, we can use a series of auto generated methods on the model
- These auto generated methods also cover manipulating associations between objects

Populating loaded models

- Assuming an entity called Parent and a dependent entity called Dependent, such that Parent.hasMany(Dependent) and a loaded parent we can then load the dependents of that particular parent by calling parent.getDependents
- We need not have defined this method as it is dynamically attached to the model
- This method functions like a find call, and thus we can pass to it the parameters we would pass to find

Auto-generated methods

```
const book = await Book.findByPk(req.params.bid)

if (book) {
 const chapters = await book.getChapters()
}
```

Handling existing models

- When dealing with legacy data, we will not be able to generate the structure of the tables in which the objects will be stored
- As such we have to make our entities match the existing structure
- We can specify the existing table name by passing

Handling existing models

```
const Student = sequelize.define('Student', {
 studentId: {
   type: Sequelize.INTEGER,
   autoIncrement: true,
   allowNull: false,
   primaryKey: true,
   field: 'student_id'  // explicit column name
in the table
 },
 firstName: {
   type: Sequelize.STRING,
   allowNull: false,
   field: 'first_name'  // explicit column name
in the table
 },
 lastName: {
   type: Sequelize.STRING,
   allowNull: false,
   field: 'last_name'  // explicit column name in the
table
 },
 emailAddress: {
   type: Sequelize.STRING,
   allowNull: false,
   validate: {
     isEmail: true,
   },
   field: 'email_address'  // explicit column name in
the table
 },
}, {
 tableName: 'students',  // explicitly specify table
name
});
```

Virtual fields

- Virtual fields are not actually stored in the database, but rather generated on the fly
- They allow for aggregate representations which might be useful the user, but would produce data duplication if stored
- They use the type VIRTUAL and must be combined with an explicit definition of a getter and setter

Virtual fields

```
const Student = sequelize.define('Student', {
 firstName: {
   type: Sequelize.STRING,
   allowNull: false,
   field: 'first_name'
 },
 lastName: {
   type: Sequelize.STRING,
   allowNull: false,
   field: 'last_name'
 },
 fullName: {
   type: Sequelize.VIRTUAL, // The type of this column is "VIRTUAL"
   get() {
     return `${this.firstName} ${this.lastName}`;
   }
 },
})
```

Falling back to native

- Unders certain circumstances, sequelize might produce inefficient queries (the n+1 select problem)
- We might also want to perform an operation which is specific to a particular RDBMS
- As such, we can always run a native query with connection.query which will return an array of records
- These can also be used similarly to prepared statements to pass and control parameters

Falling back to native

```
const sp = 'CALL concatenate_with_date(?, @result)'
const input = 'Hello, world!'

// Execute the stored procedure call
await sequelize.query(sp, { replacements: [input], type:
sequelize.QueryTypes.RAW })
const result = await sequelize.query('SELECT @result AS
result', { plain: true })
```

Lean entities

- Sequelize objects can be easily converted to JSON to serialize them
- However, sometimes we want the data content of the model without the additional functionality (we want to dehydrate the model)
- This can be achieved by passing the raw option to a sequelize query

Lean entities

```
let results = await Restaurant.findAll({
  where: { id: 1 },
  include: [
    {
      model: Menu,
      include: [{ model: MenuItem }]
    }
  ],
  raw: true
})
```

Paranoid mode

- If, for example, we want to maintain statistics which are dependent on some entity, if that entity is deleted, our stats will be invalidated
- We can opt to eschew actual deletion and use logical deletion
- If a model is said to be in paranoid mode (which can be achieved with the paranoid option set to true), whenever we try to delete it, it will be kept and instead a timestamp will be created in the column deletedAt
- All queries will ignore soft deleted records unless passed the parameter paranoid with the value false
- Note that this is not a versioning mechanism and has no impact on updates

NoSql databases

Introduction to MongoDb

What is NoSQL

- Diverse set of database management systems which do not rely on relational algebra as an underlying principle
- Approaches vary greatly
  - MongoDb is a document focused database management system
  - Redis is a data structure focused database management system
  - Neo4j is a graph database management system
  - InfluxDb is a time series database management system
- Use cases vary greatly
  - MongoDb can be used both to schemaless data, but can also be used via an application level schema
  - Redis can be used as a cache or as a message bus
  - Etc.

MongoDb principles

- NoSql DBMS focused on storing collections of documents
- Documents in a collection need not have the same schema
- Documents are stored in a flexible format called BSON (Binary jSON)
- The query language is Javascript for additional flexibility
- Extremely easy to scale horizontally
- Does not support traditional joins
- Can perform complex multi collection queries via an object pipeline called the aggregation pipeline
- Includes map/reduce support

Structure of the database

- A MongoDb database contains a series of collections
- Collections are created on demand, whenever a first document is inserted
- Collections contain a series of documents, which are can be thought as JSON objects
- The documents in a collection can have different “shapes” (they do not have to respect the same schema)
- Documents can be indexed for faster access

CRUD operations in MongoDb

- MongoDb supports the usual CRUD operations, although there are some distinctions
  - Create: insert, insertMany, in some cases update and updateMany (when functioning as upsert)
  - Read: find, findOne, aggregate
  - Update: update, updateMany
  - Delete: remove

Inserts

- Insert commands are run on a collection
- A unique ObjectID is generated for each document inserted
- The insert command inserts one record
- The insertMany command inserts an array of records
- When an update command is run with the parameter upsert: true, if the record exists it will be updated, otherwise it will be inserted

Inserts

```
var items = [0,1,2,3,4,5,6,7,8,9].map(function(e){
 return {
   name : 'student' + e,
   studentNo : e
 }
})
for (var i = 0; i < items.length; i++){
 db.students.insert(items[i])
}
var e = {
 name : 'john',
 position : 'programmer',
 salary : 1000
}
```

Simple finds

- Records can be retrieved with find and findOne
- The findOne command returns the first record matching the query
- The find command returns a cursor for all the records matching the query
- A count of all documents can be obtained with the count command
- The count command is O(n) so mongo also provides estimatedDocumentCount, which runs in constant time, but may not provide accurate results if the database is distributed
- To obtain records for which the value of an attribute is distinct we can use collection.distinct()

Finds

```
db.employees.count()

db.employees.findOne()

db.employees.find().pretty()
```

The general form of a find command

```
db.collection.find(query, projection, options)
```

- Query represents selection and provides the mechanism to define complex conditions
- Projection specifies which properties of the retrieved documents to be included in the result
- _id must be explicitly excluded, while other fields must be explicitly included
- Options provides support for limit, skip, explicit indexes, collation (what character set to be used in the query), read concen (which nodes the query can be run on in distributed architectures) as well as maximum time limits on the query

Query operators

- Query operators can be used in the condition supplied to a find command
- Various comparison, logical, bitwise operators are supported as well as specific mongo operators (geospatial, so called projection operators which allow for the navigation of the document structure etc.)
- Exhaustive list:
  - https://www.mongodb.com/docs/manual/reference/operator/query/

Query operators

```
db.employees.find({
 position : 'programmer',
 salary : {
   $gt : 1000
 }
})

db.employees.find({
 salary : {
   $gt : 1000
 }
})
```

Query operators

```
db.employees.find({$or : [
 {
   position : 'programmer',
   salary : {
     $gt : 1000
   }
 },
 {
   position : 'accountant'
 }
]})

db.employees.find({
 position : {
   $in : ['accountant', 'programmer']
 }
})
```

Updates

- Updates are performed either with the update (which updates the first matching record) or updateMany (which updates all matching records) commands
- The general form of an update command:
  - `db.collection.update(query, update, options)`
- Records matching query (which behaves like the query in find) are updated via an update expression
- Options can be used to specify if the update command behaves like an upsert, collation for the query, write concern for distributed architectures etc.

Updates

Will replace the updated record:

```
db.students.update({name : 'student5'}, {name : '55'})
```

Update operators

- Updates are performed via an operator (e.g. $inc can be used to increment the value of a field)
- Exhaustive list of update operators
  - https://www.mongodb.com/docs/manual/reference/operator/update/
- More complex transformations can be supplied as a pipeline (similar to aggregation queries)

Update operators

```
db.students.update({name : 'student6'}, {$set : {
 name : '66'
}})

db.students.updateMany({name : 'student6'}, {$set : {
 name : '66'
}})

db.employees.updateMany({},{
 $inc : {salary : 50}
})
```

Deletes

- Deletes can be performed via the delete (which deletes the first matching record) and deleteMany (which deletes all matching records) commands
- Both delete variants receive a query as a parameter (with the same format as the query for find commands)

Deletes

```
db.employees.remove({})
```

The aggregation pipeline

- MongoDb does not support the join operation that you might find in a relational database
- Complex queries can, however, be performed using the aggregation pipeline
- The aggregation pipeline is an object pipeline with a series of stages. Each record enters a stage and, after processing, passes to the next stage
- Some aggregation pipeline stages have to wait for all records, due to the nature of the operation (e.g. sorting), but in principle, records can be passed down the pipeline as soon as they are processed

Aggregation pipeline

- Complete reference
  - https://www.mongodb.com/docs/manual/reference/operator/aggregation-pipeline/
- Relevant (and free) book:
  - https://www.practical-mongodb-aggregations.com/
- Aggregation stages can refer previously calculated values by referring them with a prefixed $
- Aggregation stages can refer previously defined variables by referring them with a prefixed $$

Notable aggregation pipeline stages

- $match filters the data in order to keep only the relevant records
- $addFields adds calculated fields to the data
- $group calculates aggregated values
- $lookup plays the role join would in a relational database
- $facet allows for reuse of previous calculations in order to get multiple results
- $project can be used to flatten objects that enter the stage and

Aggregations

```
db.employees.aggregate({
 $project : {
   _id : 0,
   salary : 1,
   job : 1
 }
})

db.employees.aggregate({
 $project : {
   _id : 0,
   salary : 1,
   job : 1
 }
}, {
 $match : {
   job : 'programmer'
 }
})
```

Aggregation pipeline guidance

- $match should be used as soon as possible, as processing as few objects as possible in a stage will increase performance
- $facet should be used to parallelize related processing tasks, avoiding running two different queries (i.e. use facets when calculating a list of records and a total number of records)

Aggregations

```
db.employees.aggregate({
 $match : {
   job : 'programmer'
 }
},{
 $project : {
   _id : 0,
   salary : 1,
   job : 1
 }
},  {
 $group : {
     _id : '$job',
     avg_salary : {
       $avg : '$salary'
     },
     count : {
       $sum : 1
     }
   }
 }
)
```

Map/reduce support

- Map/reduce is an alternative processing model for distributed databases
- It functions similarly in multiple distributed NoSQL databases and is also supported by mongo
- Initially a stopgap measure since the aggregation pipeline did not work on clusters, which is no longer the case

Map/reduce

```
var items =
[0,1,2,3,4,5,6,7,8,9].map(function(e){return {name
: 'e'+e, student_number : '000' + e, grade :
_rand() * 10}})

for (var i = 0; i < items.length; i++){
 db.students.insert(items[i])
}
```


```
var items =
[0,1,2,3,4,5,6,7,8,9].map(function(e){return {name
: 'e'+e, student_number : '000' + e, grade :
_rand() * 10}})

for (var i = 0; i < items.length; i++){
 db.students.insert(items[i])
}
```


```
var grademap = function() { emit(this.name,
this.grade); };
var gradereduce = function(keyName, values) {
return Array.avg(values);};
db.students.mapReduce(grademap,gradereduce,{
out: "mapreduceout" })

db.mapreduceout.find()
```

Large file support with GridFS

- Mongo has a record limit of 16MB
- If storage of larger items is needed, GridFS can be used
- GridFS records store byte arrays, but they do not guarantee atomicity on updates to the file

Geographic data support

- Mongo supports geographic data and queries on it
- Geographic information can be represented as collections of Points
- These can be indexed via special indexes
- These collections support special operations, such as $near

Geographic data

```
db.places.insertMany( [
  {
     name: "Central Park",
     location: { type: "Point", coordinates: [ -73.97, 40.77 ]
},
     category: "Parks"
  },
  {
     name: "Sara D. Roosevelt Park",
     location: { type: "Point", coordinates: [ -73.9928,
40.7193 ] },
     category: "Parks"
  },
  {
     name: "Polo Grounds",
     location: { type: "Point", coordinates: [ -73.9375,
40.8303 ] },
     category: "Stadiums"
  }
] )
db.places.createIndex( { location: "2dsphere" } )
```

```
db.places.find(
  {
    location:
      { $near:
         {
           $geometry: { type: "Point",  coordinates: [ -73.9667, 40.78
] },
           $minDistance: 1000,
           $maxDistance: 5000
         }
      }
  }
)

db.places.aggregate( [
  {
     $geoNear: {
        near: { type: "Point", coordinates: [ -73.9667, 40.78 ] },
        spherical: true,
        query: { category: "Parks" },
        distanceField: "calcDistance"
     }
  }
] )
```

Advanced architectures

- Mongo has mechanisms for distribution of data through replica sets and collection sharded
- Disaster recovery architectures can be achieved using replica sets
- Horizontal scaling can be achieved via collection sharding
- High availability can be achieved via collection sharding but some limitations exist

Replica sets

- If integrity/availability is needed, data can be stored in ReplicaSets
- Replica Sets hold multiple copies of data and can be interacted with depending on a configurable read concern and write concern
- A replica set can only hold an odd number of nodes, although a special node, not holding data (called an arbiter node) can be used if an even number of data nodes are available

Sharded collections

- Collections can be split into multiple parts with collection sharding
- Typically, these shards are replica sets
- The shards can be then be used for geographical distribution or, if this is not needed, be used to split processing over multiple data centers

Database programming

Native programming for mongodb

Principles

- MongoDb shares a lot of concepts with Javascript
- Both the representation and the query language are compatible since the representation is BSON (serialized Js objects represented in binary form) and the query language is Javascript
- As such, beyond getting a connection, the interface of the driver is almost identical to the interface of the database itself

Getting a connection

- The mongodb package exports MongoClient to which a connection URL can be supplied
- This uses the schema mongodb://
- It can be used to specify a different port (with the default one being 27017), database as as additional options when connecting to a more complex architecture (such as a replica set) in which case it can be used to specify the levels of read and write concern

Connections

```
import { MongoClient } from 'mongodb'
const url = 'mongodb://localhost:27017'
const dbName = 'ismv4'

try {
 const client = await MongoClient.connect(url)
 console.log("Connected successfully to server")
 const db = client.db(dbName)
 console.log(db)
 client.close()
} catch (err) {
 console.log(err.stack)
}
```

CRUD operations

- Since we’re writing in Javascript and the query language for MongoDb is also Javascript, native driver CRUD operations look identical using the native driver to the operations in the console

CRUD operations

```
const insertStudent = async function(db, student) {
 try {
   const collection = db.collection('students')
   const result = await collection.insertOne(student)
   return result
 } catch (err) {
   console.log(err.stack)
 }
}

const findStudent = async function(db, name) {
 try {
   const collection = db.collection('students')
   const doc = await collection.findOne({
     name
   })
   return doc
 } catch (err) {
   console.log(err.stack)
 }
}
```

```
const updateStudent = async function(db, query, update) {
 try {
   const collection = db.collection('students')
   const result = await collection.updateOne(query, update)
   return result
 } catch (err) {
   console.log(err.stack)
 }
}

const deleteStudent = async function(db, query) {
 try {
   const collection = db.collection('students')
   const result = await collection.deleteOne(query)
   return result
 } catch (err) {
   console.log(err.stack)
 }
}
```

Database programming

ODM programming with mongoose

Principles of an ODM

- An ODM can be used to abstract the access to a document database, but it’s purpose is slightly different from an ORM
- The premise is that the database is schemaless, but we need to represent structures characteristic to a schema based database
- We can thus define persistent types
- These types can, however, be extended in time for added flexibility
- This approach gives us a programming interface similar to that of a relational database

Getting a connection

- The most popular ODM for MongoDb is mongoose which we can access from the package with the same name
- We can connect to the database with the mongoose.connect method
- One connected, whenever importing the mongoose library in a module, we will be able to access the existing connection
- This is the so called default connection

Handling errors

- The mongoose API is fully promise based, so errors can be handled via try/catch/finally blocks
- These will have MongooseError as a type if they come from mongoose or MongoError as a type if they come from the underlying driver

Getting a connection/handling errors

```
import mongoose from 'mongoose'

try {
 await mongoose.connect('mongodb://localhost:27017/ismv4')

 await mongoose.disconnect()
} catch (err) {
 console.error(err)
}
```

Multiple connections and connection pools

- We can create specific connections with mongoose.createConnection which will return the newly created connection/connection pool
- If passing the maxPoolSize parameter in the options object to createConnection we will receive a connection pool of the specified size

Defining a schema

- Schemas are defined with the constructor mongoose.Schema
- Fields can have a variety of type
- Comprehensive list of types
  - https://mongoosejs.com/docs/schematypes.html
- Notable types:
  - ObjectId allows us to create associations
  - Mixed allows us to use nested objects (subdocuments)
  - Buffer allows us to store binary data (although with a limitation in size)

Schema definition

```
// Define a schema for a menu item
const MenuItemSchema = new mongoose.Schema({
 description: String,
 price: Number
})

// Define a schema for a restaurant menu
const MenuSchema = new mongoose.Schema({
 description: String,
 items: [{
   type: mongoose.Schema.Types.ObjectId,
   ref: 'MenuItem'
 }]
})
```

Models based on schemas

- Once the schema is defined, a model can be defined on it
- The model will intermediate all operations on collections
- The created model is a constructor which allows us to obtain a document in order to perform CRUD operations on individual documents
- In contrast to Sequelize, we do not have to explicitly define relationships between models

Defining models

```
const MenuItem = mongoose.model('MenuItem', MenuItemSchema)
const Menu = mongoose.model('Menu', MenuSchema)
```

One to one relationships

- In contrast to relational databases there are two ways of defining relationships
  - Either as composition, with the parent “containing” the child (or, more precisely the id of the child)
  - Or as references, that is the child referring the parent (or, more precisely containing the id of the parent)
- One to one relationships usually in the first variant

One to many relationships

- The same two approaches apply as for one to one relationships
- If the parent contains its children this will be represented as an array of ObjectId referring a particular entity

Relationship definition

```
const RestaurantSchema = new mongoose.Schema({
 name: String,
 openingHour: String,
 closingHour: String,
 locations: [{
   type: mongoose.Schema.Types.ObjectId,
   ref: 'Location'
 }],
 menu: {
   type: mongoose.Schema.Types.ObjectId,
   ref: 'Menu'
 }
})
```

CRUD operations

- As the paradigm in which mongoose functions is extremely similar to the one mongodb itself uses, CRUD operations function extremely similarly to the ones in mongodb
- In fact, all mongodb commands can be run on the schema (when not referring to individual documents)

Inserts

- Inserts can be performed by instantiating a model and saving the resulting document with the document.save method

Inserts

```
let restaurant1 = new Restaurant({
 name: 'McDonalds',
 openingHour: '8:00 AM',
 closingHour: '10:00 PM',
 locations: [],
 menu: null
})
await restaurant1.save()
```

Updates

- Updates can be performed:
  - Either by loading a document from the database, modifying it and then using document.save to persist the changes
  - Or by using the specialized methods on the model
    - updateOne
    - updateMany
    - findOneAndUpdate
    - findOneAndReplace

Updates

```
restaurant1 = await Restaurant.findOne({ name: 'McDonalds' })
restaurant1.name = 'Mickey D\'s'
await restaurant1.save()

const location1 = new Location({
 street: '123 Main St',
 number: 1,
 city: 'New York'
})
const savedLocation1 = await location1.save()
restaurant1.locations.push(savedLocation1)
await restaurant1.save()
```

Deletes

- Deletes can be performed in two fashions:
  - Either by loading a document from the database and using document.destroy on the loaded document
  - Or by using the specialized methods on the model
    - deleteOne
    - deleteMany
    - findOneAndRemove
    - findOneAndDelete

Deletes

```
await Location.findByIdAndDelete(savedLocation3._id)
```

Queries

- Queries can be performed with
  - find
  - findById
  - findOne
- Queries receive as one of the parameters an object which is then applied as a mask to all documents in the queried collection
- The query actually returns a cursor
- On the returned cursor, sorting can be performed with the sort method
- Paging can be performed with the limit and skip methods

Queries

```
restaurant2 = await Restaurant.findOne({ name: 'Burger King' })
```

Queries

- If the records are not to be retrieved all at once due to performance constraints, the cursor itself can be obtained with the cursor method
- The cursor is an iterable object (it has a next method which returns a promise to the current record)

Populates

- By default, documents involved in relationships are lazily loaded
- In order to eager load dependent models, the document.populate method can be used
- Additionally, for any query, the populate parameter can be supplied
- Populate is supplied with one or more paths and loads the dependent documents
- If the dependent documents need to be loaded at a later moment (i.e. after the parent was loaded) the execPopulate method has to be used

Nested populates

- The populate method can also receive all arguments that a find might receive
- This includes the populate parameter, to obtain population on multiple paths and in depth

Populates

```
const restaurant = await Restaurant.findOne({ name: 'Mickey D\'s' })
.populate({
 path: 'menu',
 populate: { path: 'items' }
})

Post
 .findOne({ title: 'First Post' })
 .populate({
   path: 'comments',
   populate: { path: 'author' }
 })
```

Aggregations

- Aggregations are covered by mongoose in a similar fashion to the native driver
- Given a starting model, the aggregate method can be called on the model, receiving as a parameter the aggregation pipeline
- All the aggregation steps and options are supported in mongoose in the same fashion as in the native driver or, indeed, the command line interface

Aggregate

```
const pipeline = [
  {
    $match: { name: 'Mickey D\'s' }
  },
  {
    $lookup: {
      from: 'menus',
      localField: 'menu',
      foreignField: '_id',
      as: 'menu'
    }
  },
  {
    $unwind: '$menu'
  },
  {
    $lookup: {
      from: 'menuitems',
      localField: 'menu.items',
      foreignField: '_id',
      as: 'items'
    }
  },
  {
    $project: {
      _id: 0,
      items: 1
    }
  },
  {
    $facet: {
      items: [
        { $unwind: '$items' },
        { $replaceRoot: { newRoot: '$items' } }
      ],
      count: [
        { $group: { _id: null, count: { $sum: { $size: '$items' } } } }
      ],
      [ { $project: { _id: 0 } } ]
    }
  }
]

// Execute the aggregation pipeline
const result = await Restaurant.aggregate(pipeline)
```

Transactions

- MongoDb supports transactions
- On the mongoose default connection, a transaction can be started as part of a session
- A session is created with mongoose.startSession
- Transactions can be performed via the session.withTransaction method which receives a function (which performs the operations in the transaction) as a parameter
- If the function runs successfully the transaction will be committed

Lean entities

- Objects returned by the finders mongoose exposes are complex, containing a series of methods
- If we prefer to receive a result containing only the data, we can postfix any query with the lean method, which dehydrates the models

Lean

```
const posts = await Post.find().lean()

const posts = await Post.find()
 .populate({
   path: 'comments',
   populate: { path: 'author' }
 })
 .populate('author')
 .lean()
```

Hybrid fields

- One of the advantages using mongoose has is that we can evolve our data model in time
- Until it is necessary to define additional schemas/models, we can hold intermediary data in a mixed field
- A mixed field is a Javascript object, not handled directly by mongoose
- Since mongoose does not monitor the field for changes, if a mixed field is the only modified in an object, when calling the save method it will not actually be updated
- To avoid this, when only updating a mixed field we can call model.markModified to instruct mongoose to perform the update

Virtual fields

- Virtual fields are not stored in the database, but are constructed when requested
- This is done to balance the need for development convenience in the applications accessing the database with the need to not store redundant data

Virtual fields

```
const userSchema = new Schema({
 firstName: String,
 lastName: String
}, {
 toJSON: { virtuals: true },
 toObject: { virtuals: true }
})

userSchema.virtual('fullName').get(function () {
 return this.firstName + ' ' + this.lastName
})

const User = mongoose.model('User', userSchema)
```

```
await
mongoose.connect('mongodb://localhost:27017/ismv
4')

const user = new User({ firstName: 'John',
lastName: 'Doe' })
await user.save()
console.log(user.fullName)

await mongoose.disconnect()
```

Populated virtual fields

```
const UserSchema = new Schema({
 firstName: String,
 lastName: String
}, {
 toJSON: { virtuals: true },
 toObject: { virtuals: true }
})

UserSchema.virtual('tags', {
 ref: 'Tag',
 localField: '_id',
 foreignField: 'forResource',
 justOne: false
})

const TagSchema = new mongoose.Schema({
 forResource: {
   type: mongoose.Schema.Types.ObjectId,
   required: true,
   refPath: 'onModel'
 },
 tagName: String
}, {
 timestamps: true
})
```

```
const User = mongoose.model('User', UserSchema)
const Tag = mongoose.model('Tag', TagSchema)

let user = new User({
 firstName: 'John',
 lastName: 'Smith'
})
await user.save()

const tag = new Tag({
 tagName: 'special',
 forResource: user
})
await tag.save()

user = await User.findById(user._id).populate({
path: 'tags' }).lean()
console.log(user)
```

Redis/Redis programming

General principle

- Redis is a distributed data structure store
- In contrast to more traditional databases, Redis stores a series of keys, which each key having a type
- For example a key might be a list of strings and supports the usual list operations
- The primitive values in Redis are strings

Getting a connection/handling errors

```
try {
 // Create a Redis client
 const client = redis.createClient()
 await client.connect()
  await client.quit()
} catch (error) {
 console.warn(error)
}
```

Supported data structures (core)

- String - sequences of bytes
- List - lists of strings sorted in the order of insertion
- Set - sets of unordered unique strings
- Hash - record types represented as lists of key/value pairs
- Zset (Sorted Set) - sets of strings sorted by a score associated with each string

Supported data structures

- Stream - represent append only structures similar to a read-only list
- Geospatial index - special structures for geographic operations
- Bitfield - record multiple counter values in a string
- Bitmap - allow for bitwise operations on string representations
- Hyperloglog - structure that allows probabilistic counting (cardinality)

Strings

- The most basic data-type
- Useful for caching
- Support expiration
- Have a limit of 512MB
- If they can be converted (automatically) to integers or floats, they support additional operations (incrementing the value)

String commands

```
await client.set('user:name:1', 'jim')
 await client.set('user:counter:1', 0)
 await client.incr('user:counter:1')
 let result = await client.getSet('user:name:1', 'john')
 console.log(result)
 await client.append('user:name:1', ' smith')
 result = await client.get('user:name:1')
 console.log(result)
 let len = await client.strLen('user:name:1')
 result = await client.getRange('user:name:1', 5, len)
 console.log(result)
```

Lists

- Lists are singly linked-lists
- Can be used as stacks or queues
- Can support blocking operations for synchronization between aplications

List commands

```
await client.lPush('asset:1', 'a')
await client.lPush('asset:1', 'b')
await client.lPush('asset:1', 'c')
await client.lPush('asset:1', 'd')
await client.lPush('asset:1', 'e')
let result = await client.lPop('asset:1')
console.warn(result)
result = await client.lRange('asset:1', 2, 1)
console.warn(result)
const id = uuid()
await client.lSet('asset:1', 2, id)
await client.lRem('asset:1', 1, id)
const len = await client.lLen('asset:1')
const remaining = await client.lRange('asset:1', 0, len)
console.warn(remaining)
```

Sets

- Sets hold unique values with deterministic uniqueness
- Redis supports operations both within sets (e.g. checking if a value is a member of a set) and between sets (e.g. check which values are common between two sets)

Set commands

```
await client.sAdd('assets:1', '1')
await client.sAdd('assets:1', '2')
await client.sAdd('assets:1', '3')
await client.sAdd('assets:2', '1')
await client.sAdd('assets:2', '5')
await client.sAdd('assets:2', '7')
let result = await client.sCard('assets:1')
result = await client.sInter(['assets:1', 'assets:2'])
console.log(result)
result = await client.sMembers('assets:1')
console.log(result)
result = await client.sMembers('assets:2')
console.log(result)
let cursor = await client.sScan('assets:1', 0)
console.log(cursor)
for (const item of cursor.members) {
  console.log(item)
}
```

Hashes

- Hashes in Redis are similar to hashes as defined in various programming languages
- They allow us to store multiple properties in a single key
- As with list they support additional commands for fields which can be converted to int and float

Hashes

```
await client.hSet('user:1', 'username', 'john.smith')
await client.hSet('user:1', 'type', 'admin')
let result = await client.hGetAll('user:1')
console.log(result)
result = await client.hKeys('user:1')
console.log(result)
result = await client.exists('user:1', 'type')
console.log(result)
result = await client.hVals('user:1')
console.log(result)
```

Zsets

- Zsets (Sorted sets) store unique values alongside a priority value
- The values in a set are sorted according to the priorities
- Supports a series of additional operations over regular sets
- Supports blocking operations and set member copying

ZSet - producer

```
while (!done) {
  const value = await askQuestion('Value/prio: ')
  if (value.trim() === 'quit') {
    break
  }
  const [item, priority] = value.trim().split(' ')
  console.log(item)
  console.log(priority)
  await client.zAdd('tasks:1', [{ score: priority, value: item}])
}
```

ZSet - consumer

```
while (true) {
  const task = await client.bzPopMax('tasks:1', 10)
  if (task) {
    console.log('Received task:')
    console.log(task)
  }
}
```

Pubsub

- Redis offers a publish-subscribe mechanism
- This is the next step in complexity to just watching lists or zsets
- A subscriber can watch a particular topic
- A publisher publishes messages to a watched topic
- The mechanism is somewhat volatile, in the sense that a log of the messages is not kept

Pubsub producer

```
let done = false
while (!done) {
  const value = await askQuestion('task: ')
  if (value.trim() === 'quit') {
    break
  }
  client.publish('tasks:1', value)
}
```

Pubsub consumer

```
await client.subscribe('tasks:1', (task) => {
  if (task) {
    console.log('Received task:')
    console.log(JSON.parse(task))
  } else {
    finished = true
  }
})

while (!finished) {
  const value = await askQuestion('Write "quit" to exit: ')
  if (value.trim() === 'quit') {
    break
  }
}
```

Streams

- Redis streams are append only data structures
- They can be used as being fixed size so as not to infinitely grow
- Streams can be used to distribute processing to multiple consumers, just like publish-subscribe
- The advantage of streams is a more flexible interface, in the sense that the messages are kept, client acknowledgements are supported etc

Other structures

- Bitmaps
- Bitfields
- Geospatial indexes
- Hyperloglogs
- Extended even further in redis-stack